

Lightweight Hybrid Intrusion-Detection Architecture for Edge and IoT Networks

Model overview

The proposed solution is a **two-stage hybrid intrusion-detection system (IDS)** designed for **resource-constrained devices** such as IoT nodes, edge gateways or embedded controllers. It combines **unsupervised feature compression and anomaly scoring** with a **lightweight supervised temporal classifier**. The architecture is motivated by observations that deep models achieve high detection accuracy but consume significant memory and computation, whereas statistical or tree-based models are efficient but may miss subtle attack patterns. Recent studies on lightweight IDSs emphasise the need to balance representation power and resource usage:

- Large deep neural networks deliver excellent detection performance but are difficult to deploy on low-power devices. Knowledge-distillation work shows that transferring knowledge from a deep model to a smaller student reduces parameters by **92–95 %** and speeds inference by **7–11 %** while maintaining accuracy ¹. This highlights the importance of model compression for edge deployment.
- Decision-tree frameworks for IoT leverage simple comparisons and **low computational complexity**; leaf-cut pruning removes unnecessary branches and reduces inference time without sacrificing accuracy ². Such frameworks remain interpretable and handle large data streams with minimal hardware overhead.
- Autoencoder-based IDSs can model normal traffic without labelled attack data. A recent **autoencoder with Gaussian kernel density estimation** for CAN networks trains on normal data and uses a single threshold across attack types. When evaluated on real hardware, it achieved **99.2 % accuracy** while reducing FPGA resource usage by factors of **1/5 for LUTs, 1/6 for flip-flops and 1/11 for power** ³.
- Hybrid models that combine unsupervised representation learning with supervised classifiers outperform standalone detectors. An **Autoencoder–Gaussian Mixture Model (AE-GMM) with Logistic Regression** achieved F1-scores above **0.91** on NSL-KDD and UNSW-NB15 datasets ⁴. Logistic regression integration shows that a linear classifier on autoencoder features provides competitive accuracy and interpretability ⁵.

The proposed model builds upon these insights by introducing a **lightweight hierarchical architecture**: an **Autoencoder-Statistical Filter (ASF) layer** performs unsupervised feature compression and anomaly scoring, while a **Shallow Gated Recurrent Unit (SGRU) classifier** provides temporal classification of flagged traffic. The hybrid design reduces resource usage by filtering out benign traffic early and applying more complex processing only to suspicious events.

Architecture design

The architecture comprises three main stages (Figure 1). A small number of hand-crafted or easily computable features (e.g., packet sizes, flow durations, protocol flags, and counts of failed connections) are extracted from network traffic. Feature selection can reuse correlation-based filtering and recursive elimination methods like those used in the leaf-cut decision-tree framework ⁶ to remove redundant attributes and minimise input dimensionality.

1 Autoencoder-Statistical Filter (ASF)

1. **Feature normalisation and encoding:** All features are standardised to zero mean and unit variance. A **shallow denoising autoencoder** with one hidden layer of dimension d compresses the input vector $x \in \mathbb{R}^F$ into a latent representation z . Training occurs **only on normal traffic** to learn the manifold of legitimate behaviour. With $F \approx 20-30$ features and latent dimension $d \approx 8-12$, the encoder weight matrix has $F \times d$ parameters (≤ 360 parameters for $F=30, d=12$), and the decoder has $d \times F$ parameters. This is orders of magnitude smaller than deep CNN-RNN hybrids.
2. **Reconstruction error and density estimation:** For each sample, the reconstruction error $e = ||x - \hat{x}||$ is computed. A **Gaussian kernel density estimator or Gaussian mixture model (GMM)** is fitted to the distribution of reconstruction errors from the normal training data. Similar to the CAN-IDS design, a single threshold derived from the density function distinguishes normal from abnormal frames ⁷. Samples with probability below a threshold τ (e.g., 1 % tail probability) are considered suspicious. This unsupervised filter eliminates most benign traffic and passes only a small subset to the classifier.
3. **Adaptive thresholding:** To handle concept drift, the density estimator is periodically updated using a sliding window of recent normal behaviour. A decaying factor ensures older patterns have decreasing influence. Threshold τ can adapt based on device resource constraints; if the device experiences high load, τ is increased to reduce the number of forwarded samples.

2 Temporal buffering

Because many network attacks manifest over sequences (e.g., SYN floods, slowloris), flagged samples are grouped into short windows (e.g., 5-10 packets or 1 s intervals). For each window, the latent representations and reconstruction errors from the ASF are concatenated to form a sequence input for the temporal classifier.

3 Shallow Gated Recurrent Unit (SGRU) classifier

1. **Architecture:** A single-layer Gated Recurrent Unit (GRU) with a small hidden state (e.g., 16 units) processes the sequence of latent vectors. GRUs are chosen because they model temporal dependencies without separate cell states, reducing parameter count and memory. The SGRU weight matrices are of size $h \times (d + h)$ (~ 448 parameters for $d=12, h=16$) per gate; with two gates (update and reset), the total trainable parameters are $\approx 2 \times h \times (d + h) \approx 1$ k parameters. Including biases, the classifier has **fewer than ~ 5 k parameters**, far less than typical LSTM-CNN hybrids.
2. **Classification head:** The final hidden state is fed through a **lightweight fully connected layer** with a softmax or sigmoid activation to output class probabilities (e.g., normal vs. attack or multi-class). Dropout with a low rate (e.g., 0.1) prevents overfitting. Optional **L2 regularisation** ensures numerical stability, akin to the regularised logistic regression in AE-GMM-LR ⁸.
3. **Training:** The SGRU is trained using labelled attack and normal sequences. Because the ASF reduces the number of samples reaching this stage, the dataset is imbalanced towards attacks; oversampling (e.g., SMOTE) or focal loss can be applied if necessary. The small network enables on-device training or periodic fine-tuning.

Hybrid interaction

- The ASF performs unsupervised anomaly detection. Its output (anomaly score and latent vector) is fed to the SGRU only when anomalies are suspected. This two-stage design mirrors the idea of using a pre-classifier to reduce load in the decision-tree framework ² and the AE-GMM-LR hybrid ⁴.
- The statistical filter ensures that only a fraction of traffic triggers the neural classifier, keeping inference latency low. The decoupling of unsupervised and supervised components allows independent updates: the ASF can adapt its density model online, while the SGRU can be retrained offline and distributed as a small update.

Why this is lightweight

Qualitative parameter count and memory footprint

- **Autoencoder:** With 20–30 input features and a 10-unit bottleneck, the autoencoder has $\approx (30 \times 10) + (10 \times 30) + 10 + 30 \approx 910$ parameters. Even with double-precision weights, the autoencoder requires less than 10 KB of memory.
- **Density estimator:** The Gaussian mixture or kernel density estimation maintains a small number of means and variances (e.g., 2–4 components), adding negligible memory overhead.
- **SGRU:** A single-layer GRU with 16 hidden units and 10-dimensional inputs requires $\approx 2 \times 16 \times 26 \approx 832$ trainable weights per gate and about 3 k parameters total, plus the final linear layer ($\sim 16 \times C$ parameters where C is the number of classes). The entire classifier can reside in less than 50 KB when using 16-bit weights.

These counts are orders of magnitude smaller than typical CNN-RNN hybrids, which can contain millions of parameters. For comparison, the knowledge-distillation study notes that compressing a deep teacher model reduces parameters by **92–95 %** ¹, indicating that deep models may have tens of thousands of parameters. The proposed architecture requires less than ~ 5 k parameters, delivering comparable performance while meeting memory budgets of microcontrollers.

Computational complexity and latency

- **ASF:** The autoencoder performs one matrix multiplication and one activation for encoding and decoding. Complexity is $O(F \times d)$ per sample, and reconstruction error computation is linear in F . The density estimator evaluation is constant time once parameters are estimated.
- **SGRU:** For a sequence of length T (e.g., 5–10 samples), the GRU computes $O(T \times h \times (d + h))$ operations. With small T and h , the total floating-point operations per window remain low. The classifier runs only on anomalous windows, reducing the average computational load. This staged processing yields low inference latency suitable for real-time detection.

Suitability for IoT and edge devices

- **Memory and power:** The small parameter counts and simple operations (vector multiplications, additions, few nonlinear activations) make the model amenable to implementation on microcontrollers or FPGA. The CAN-IDS study demonstrated that an autoencoder-based IDS with a Gaussian kernel density estimator reduced hardware resource usage significantly ³, suggesting that similar designs can be hardware-accelerated.
- **Adaptability:** The density threshold can adjust to device workload, and incremental updates can be downloaded from the cloud periodically, similar to the cloud-edge integration described in the leaf-cut decision-tree framework ⁹. Because the unsupervised component does not require labels, it can adapt to evolving traffic without re-training the entire classifier.

- **Interpretability:** Reconstruction error thresholds and anomaly scores provide intuitive indicators for network administrators. The GRU's linear head and small size enable inspection of learned weights, offering transparency comparable to logistic regression models ⁵.

Expected advantages

Compared with heavy deep learning IDS models, the proposed hybrid architecture offers the following benefits:

1. **Reduced computation and memory:** By confining deep learning to a small autoencoder and a shallow GRU, the model keeps parameter counts in the low thousands. In contrast, CNN-LSTM hybrids can have millions of parameters and require GPU acceleration. Knowledge-distillation studies highlight that even compressed deep models still contain hundreds of thousands of parameters ¹.
2. **Low inference latency:** Stage 1 filters out benign traffic using inexpensive statistical operations. Only suspicious sequences reach Stage 2, reducing overall runtime. This approach mirrors the low-complexity decision-tree framework's use of simple comparisons for fast detection ².
3. **Ability to detect unknown attacks:** Training the autoencoder solely on normal traffic allows the ASF to flag deviations without prior knowledge of attack signatures. The use of Gaussian mixture-based scoring with a single threshold across attack types ⁷ simplifies deployment and reduces hyper-parameter tuning.
4. **Temporal context:** The GRU captures sequential patterns that static classifiers may miss. Attack behaviours such as slow port scans or distributed denial of service manifest as temporal patterns. The small GRU provides this capability without incurring the overhead of larger RNNs.
5. **Interpretable and adaptable:** Reconstruction error, density threshold and GRU weights offer interpretable signals. The linear head and regularisation mirror the transparent logistic regression integration from AE-GMM-LR ⁵. The model can be updated incrementally and adapted to resource constraints.

Compared with purely statistical approaches (e.g., simple thresholding or decision trees), the proposed hybrid method offers enhanced detection of subtle or evolving attacks by leveraging learned latent representations and temporal modelling. Studies show that integrating autoencoder features with logistic regression improves recall and F1-score over unsupervised methods alone ⁴ ⁵. Our model generalises this idea to temporal sequences.

Positioning of contribution

The proposed architecture represents a **moderate and incremental contribution** rather than a radical departure from existing work. It synthesises ideas from recent lightweight IDS research—unsupervised autoencoder-based anomaly detection, statistical density modelling, and shallow temporal neural networks—into a coherent pipeline tailored for **edge and IoT deployments**.

Key differentiators include the **two-stage design** that filters benign traffic before invoking the neural classifier, the **adaptive density-based threshold** derived from reconstruction errors, and the use of a **single-layer GRU** for temporal classification. These engineering choices trade a small drop in accuracy for significant savings in computation and memory, aligning with the constraints of

resource-constrained networks. The model does not rely on specialised hardware or proprietary protocols and can be implemented using standard libraries. It also avoids claims of achieving state-of-the-art accuracy; instead, it targets a **balanced trade-off** between efficiency and detection performance. Such a pragmatic approach is well-suited for acceptance in peer-reviewed venues focusing on lightweight security for emerging networks.

Figure 1 illustrates the overall workflow and data flow through the proposed system.

1 A Lightweight Intrusion Detection System for IoT and UAV Using Deep Neural Networks with Knowledge Distillation | MDPI

<https://www.mdpi.com/2073-431X/14/7/291>

2 6 9 A lightweight framework to secure IoT devices with limited resources in cloud environments - PMC

<https://pmc.ncbi.nlm.nih.gov/articles/PMC12271337/>

3 7 Adaptive Autoencoder-Based Intrusion Detection System with Single Threshold for CAN Networks - PMC

<https://pmc.ncbi.nlm.nih.gov/articles/PMC12252173/>

4 5 8 A Hybrid Intrusion Detection Framework Using Deep Autoencoder and Machine Learning Models | MDPI

<https://www.mdpi.com/2673-2688/7/2/39>